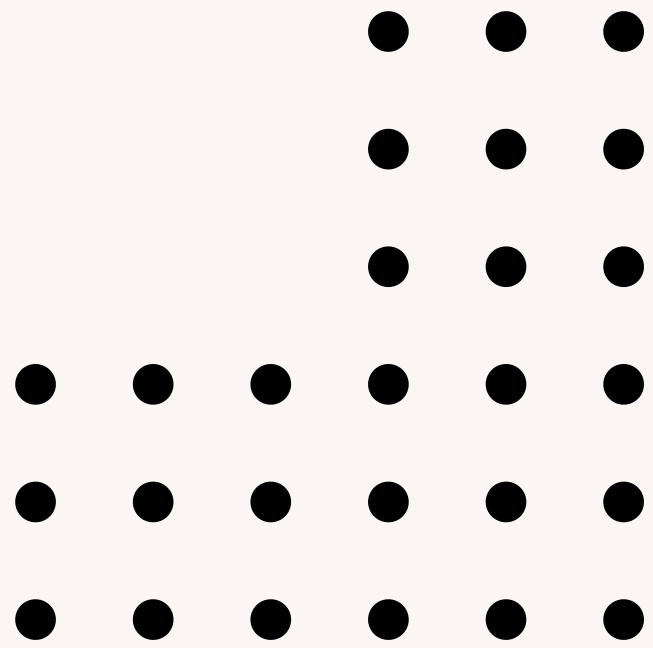
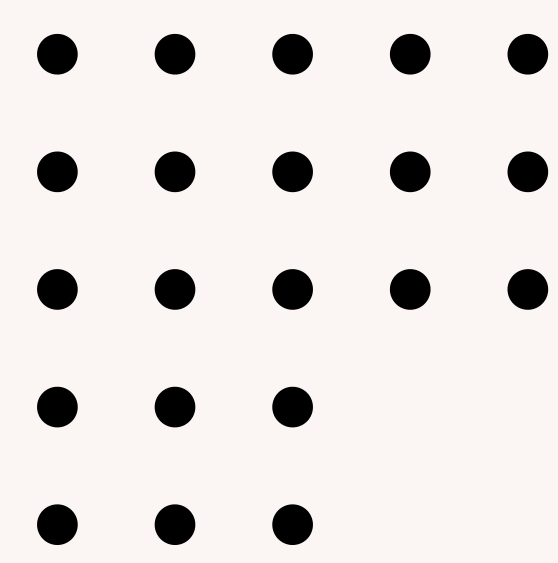
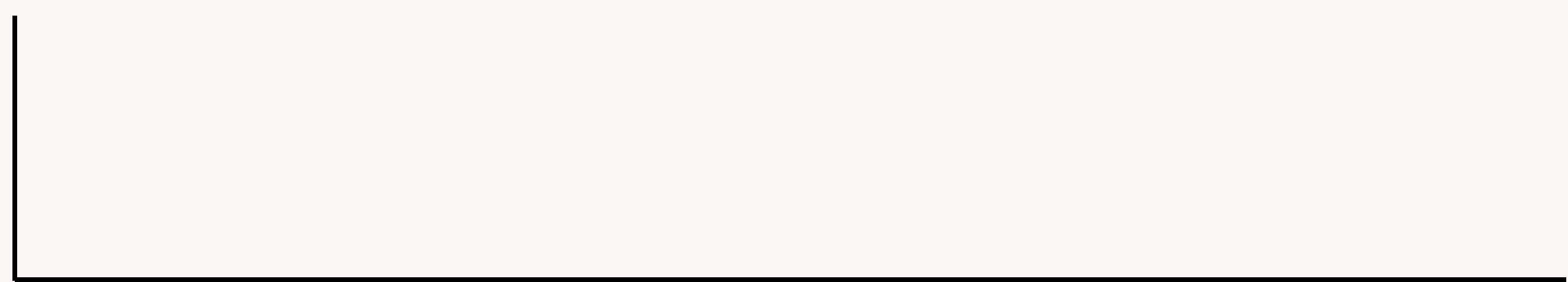
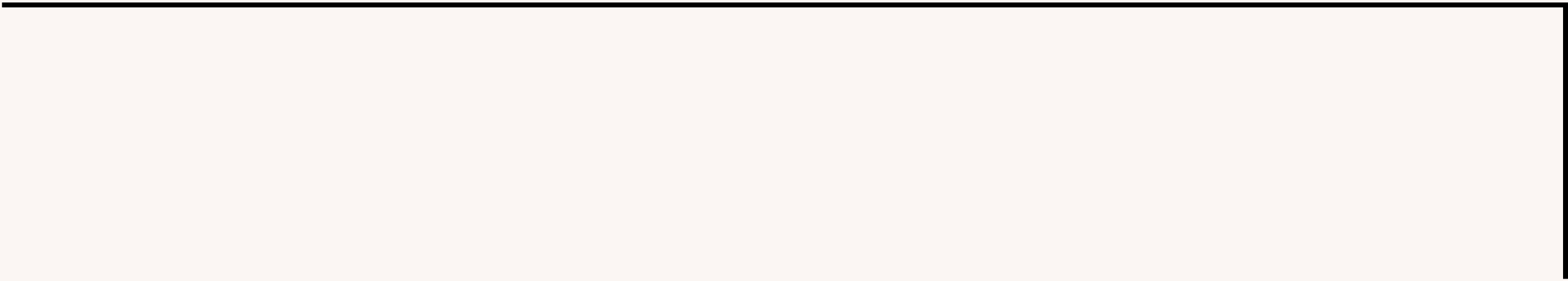


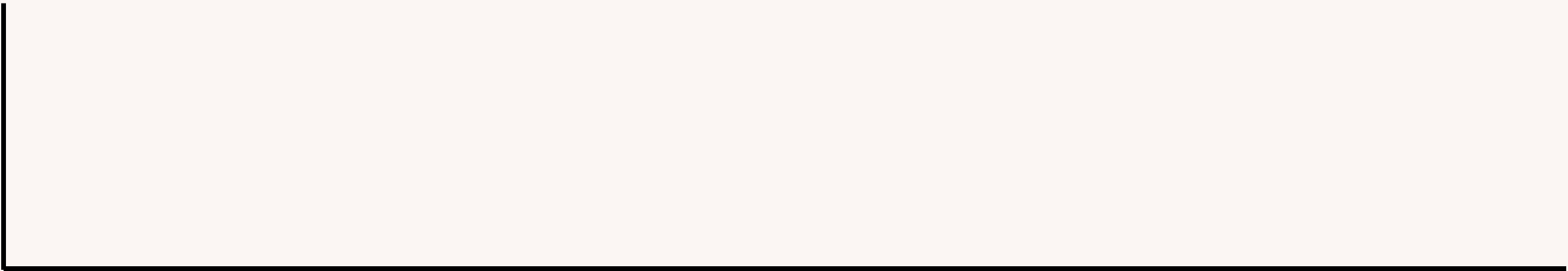
AULA 07

Maratona de programação Unioeste





A . TORNÊIO DE TENIS



A prefeitura contratou um novo professor para ensinar as crianças do bairro a jogar tênis na quadra de tênis do parque municipal. O professor convidou todas as crianças do bairro interessadas em aprender a jogar tênis. Ao final do primeiro mês de aulas e treinamentos foi organizado um torneio em que cada participante disputou exatamente seis jogos. O professor vai usar o desempenho no torneio para separar as crianças em três grupos, de forma a ter grupos de treino em que os participantes tenham habilidades mais ou menos iguais, usando o seguinte critério:

- participantes que venceram 5 ou 6 jogos serão colocados no Grupo 1;
- participantes que venceram 3 ou 4 jogos serão colocados no Grupo 2;
- participantes que venceram 1 ou 2 jogos serão colocados no Grupo 3;
- participantes que não venceram nenhum jogo não serão convidados a continuar com os treinamentos.

Dada uma lista com o resultado dos jogos de um participante, escreva um programa para determinar em qual grupo ele será colocado.

Input

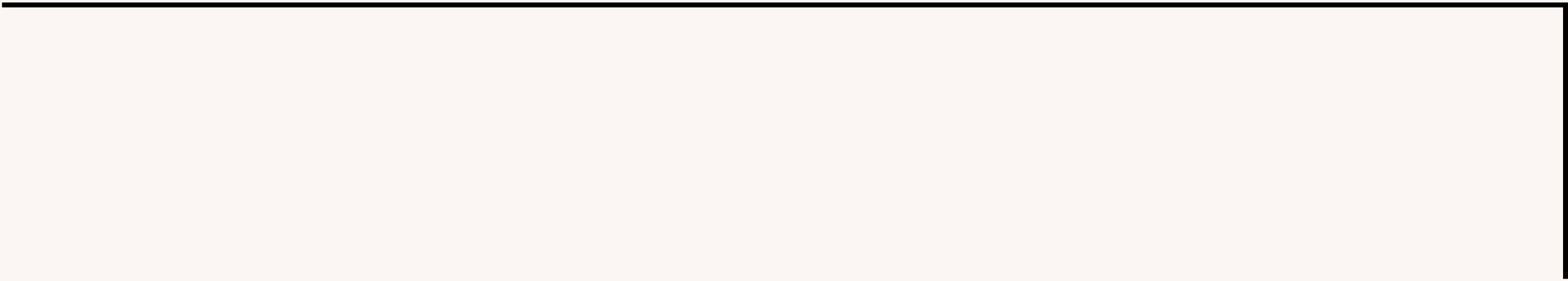
A entrada consiste de seis linhas, cada linha indicando o resultado de um jogo do participante. Cada linha contém um único caractere: V se o participante venceu o jogo, ou P se o jogador perdeu o jogo. Não há empates nos jogos.

Output

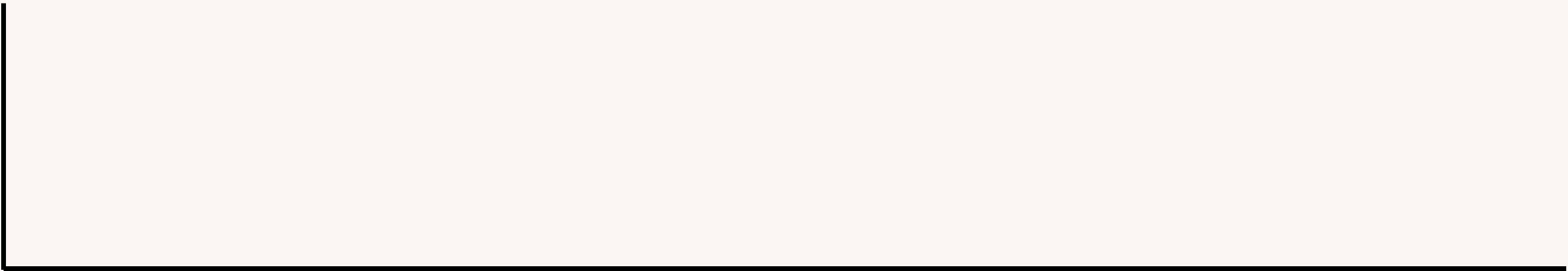
Seu programa deve produzir uma única linha na saída, contendo um único inteiro, identificando o grupo em que o participante será colocado. Se o participante não for colocado em nenhum dos três grupos seu programa deve imprimir o valor -1.

Examples	
input	
P	
P	
V	
P	
P	
P	
output	
3	

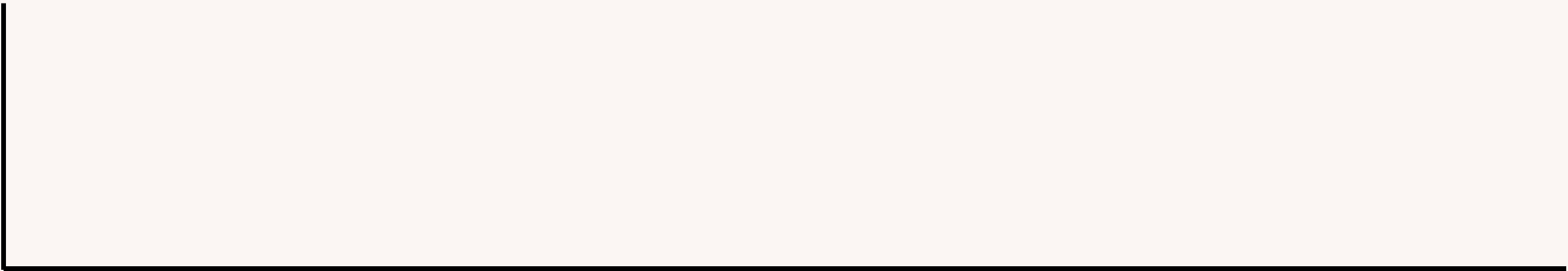
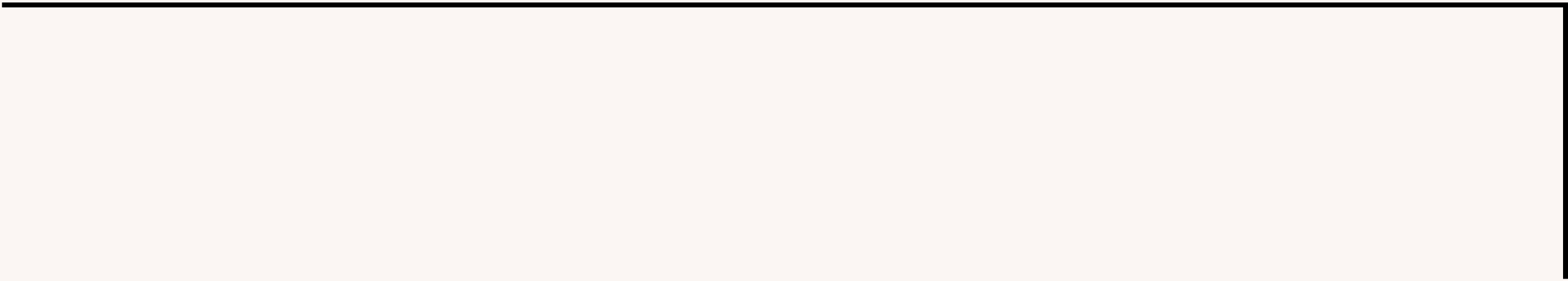
input
P
P
P
P
P
P
output
-1



CONTAR A QUANTIDADE DE 'V'




```
signed main(){
    winton;
    int ans = 0;
    for (int i = 0; i < 6; i++){
        char a;
        cin >> a;
        if (a == 'V')ans++;
    }
    if (ans == 0) cout << "-1" << endl;
    else if (ans > 0 && ans < 3) cout << "3" << endl;
    else if (ans > 2 && ans < 5) cout << "2" << endl;
    else cout << "1" << endl;
}
```



B . CONTAS A PAGAR

Vô João está aposentado, tem boa saúde, mas a vida não está fácil. Todo mês é um sufoco para conseguir pagar as contas! Ainda bem que ele é muito amigo dos donos das lojas do bairro, e eles permitem que ele fique devendo. Depois de pagar aluguel, conta de luz, conta de água, conta do telefone celular e conta do mercado, Vô João ainda tem que pagar as contas do Açougue, da Farmácia e da Padaria. Dados o valor que Vô João tem disponível e o valor das contas do Açougue, Farmácia e Padaria, escreva um programa para determinar quantas contas, entre as três que ainda não foram pagas, Vô João consegue pagar

Input

A entrada contém quatro linhas. A primeira linha contém um inteiro V , o valor que Vô João tem disponível para pagar as contas. A segunda linha contém um inteiro A , o valor da conta do Açougue. A terceira linha contém um inteiro F , o valor da conta da Farmácia. A quarta linha contém um inteiro P , o valor da conta da Padaria.

Output

Seu programa deve produzir uma única linha, contendo um único inteiro, o maior número de contas que Vô João consegue pagar.

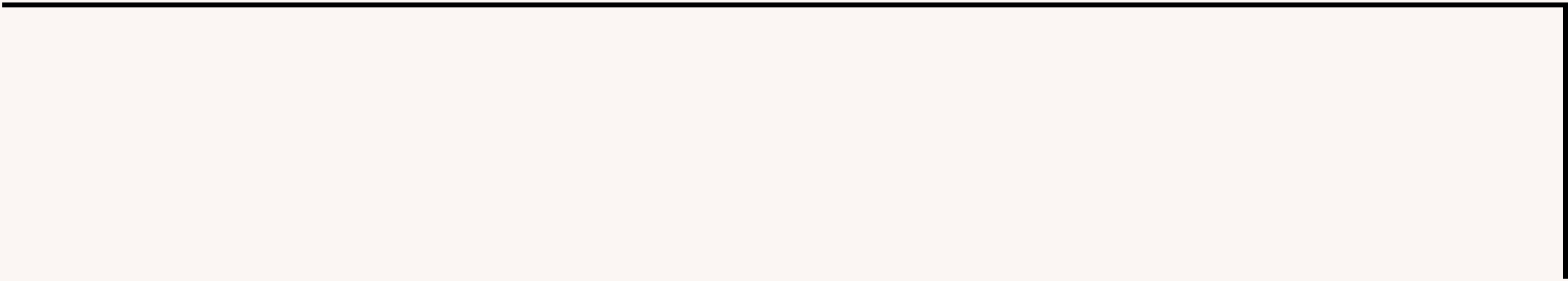
Restrições

- $0 \leq V \leq 2000$
- $1 \leq A \leq 1000$
- $1 \leq F \leq 1000$
- $1 \leq P \leq 1000$

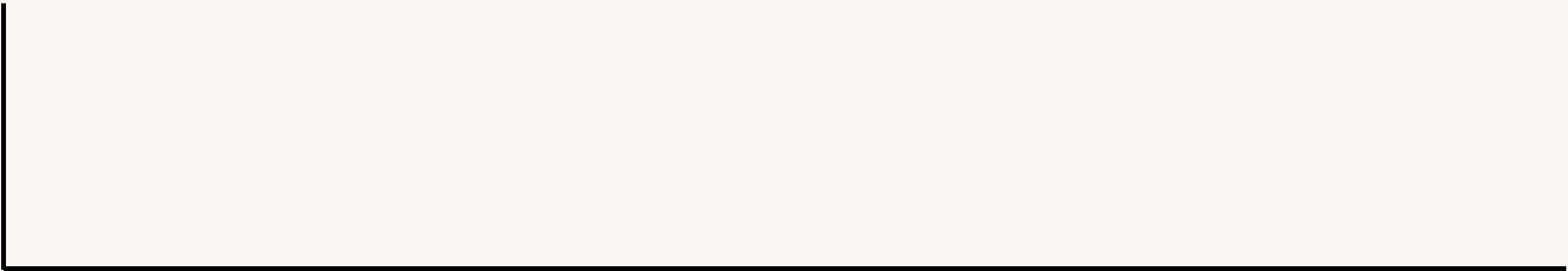
input
200
100
180
90
output
2

input
50
82
99
51
output
0

input
100
30
40
30
output
3



**CONTAR QUANTOS NÚMEROS
SOMADOS CABEM EM V**



input

200

100

180

90

output

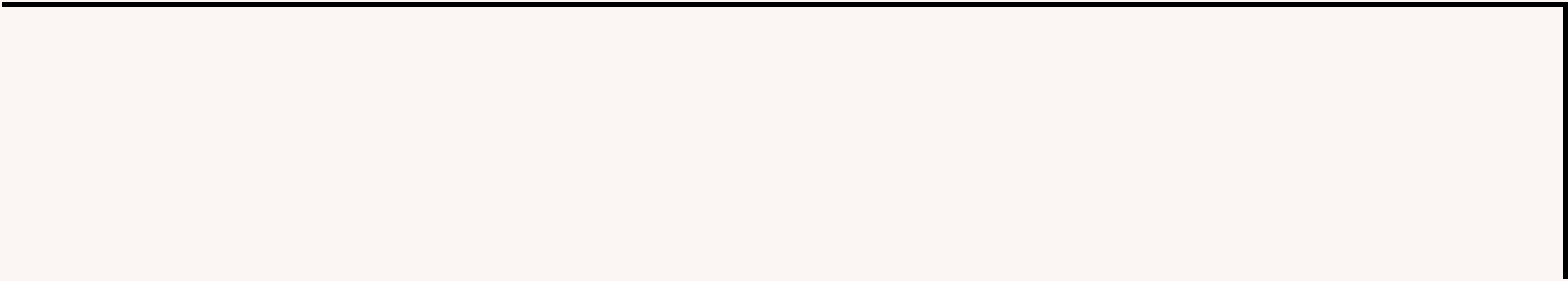
2

$$200 - 100 = 100$$

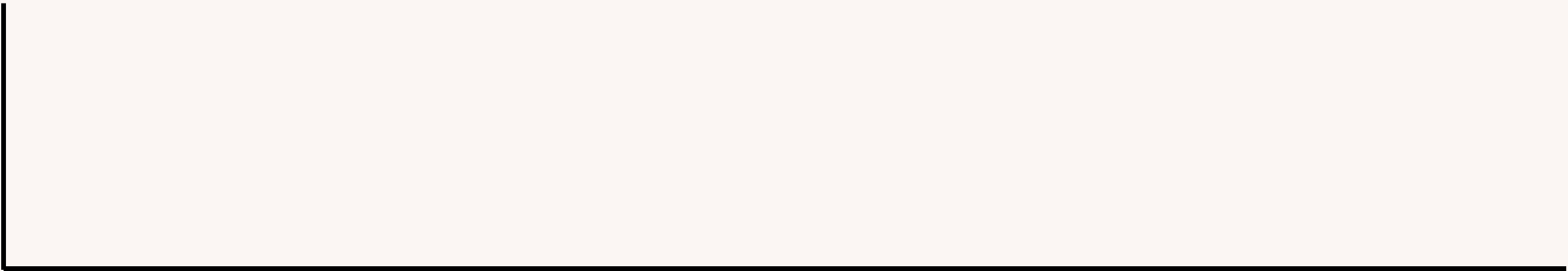
$$100 - 180 =$$

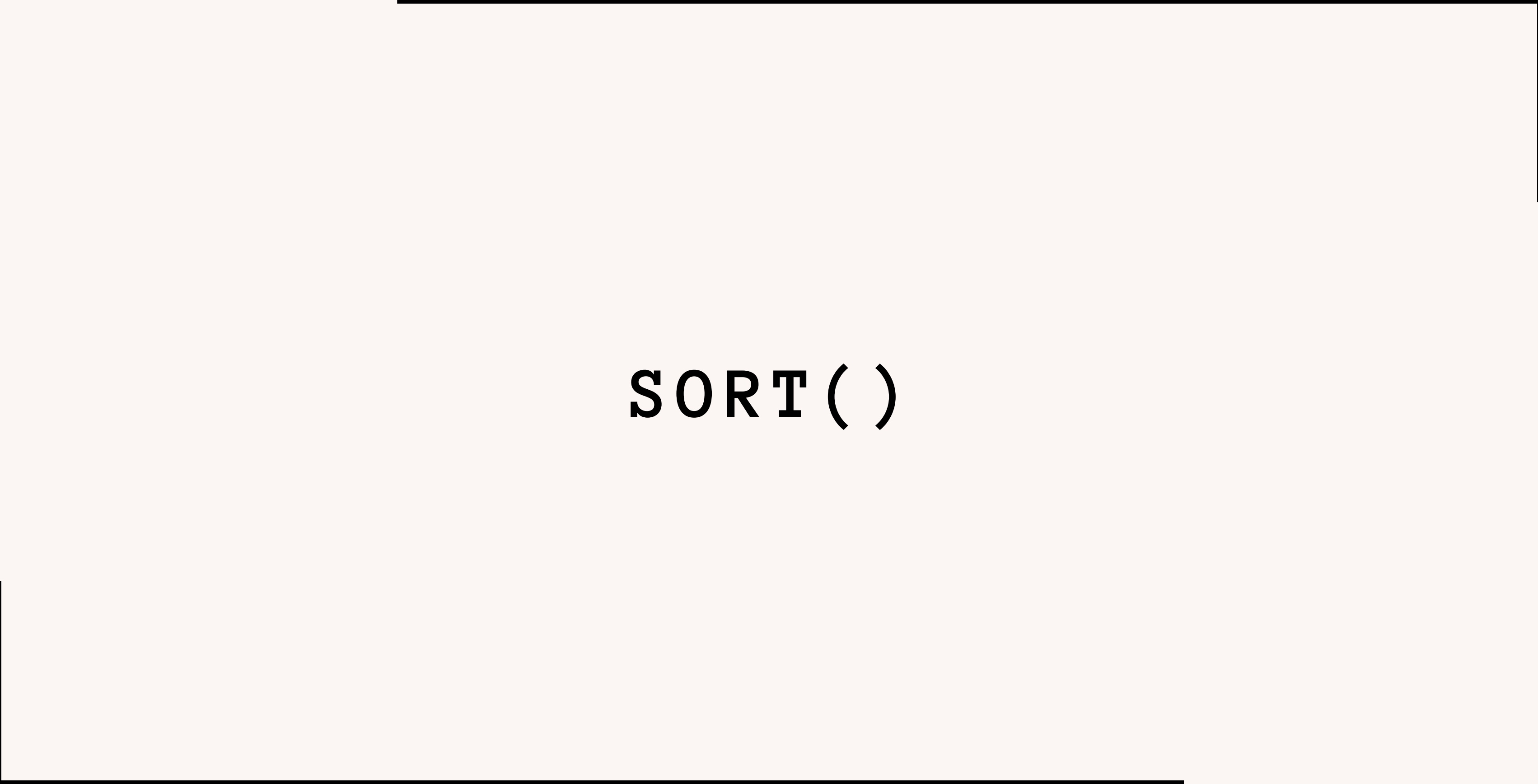
$$200 - 90 = 110$$

$$110 - 100 = 10$$



**É SEMPRE ÓTIMO ORDENAR OS
VALORES DO MENOR PARA O MAIOR**





SORT ()


```
VECTOR<INT> A = {5, 8, 3, 6, 2}
```

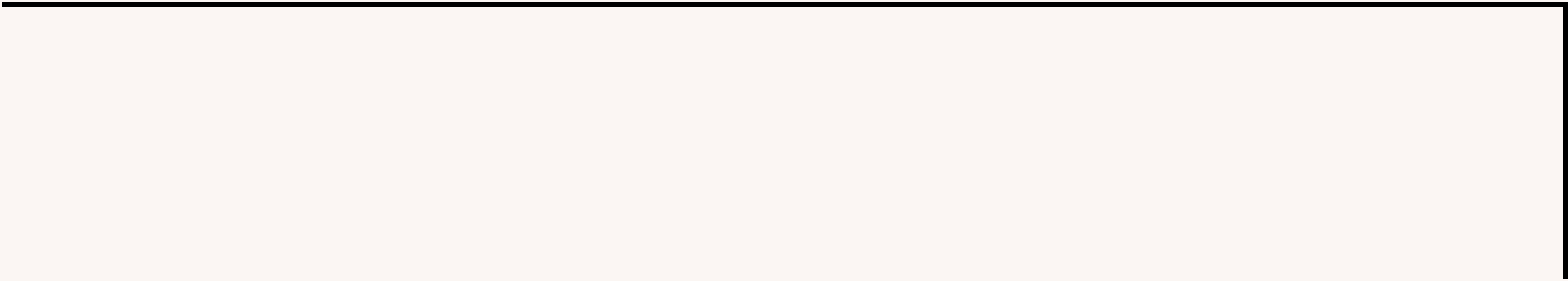
SORT()



```
VECTOR<INT> A = {2, 3, 5, 6, 8}
```

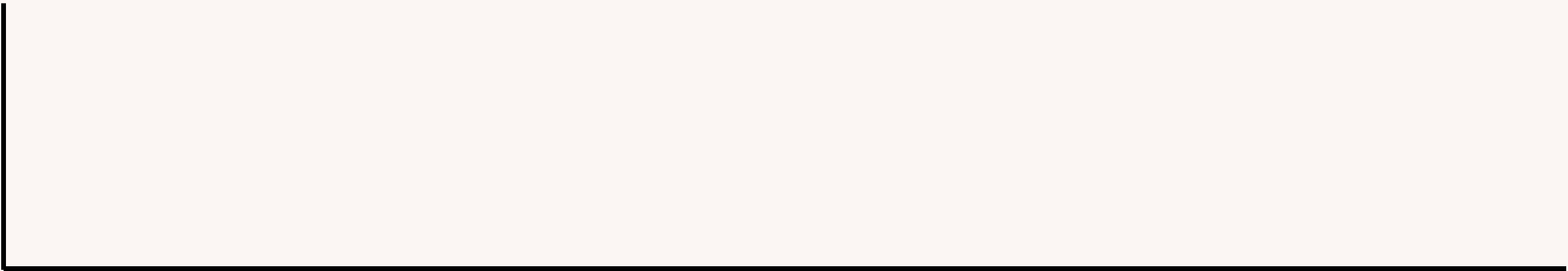


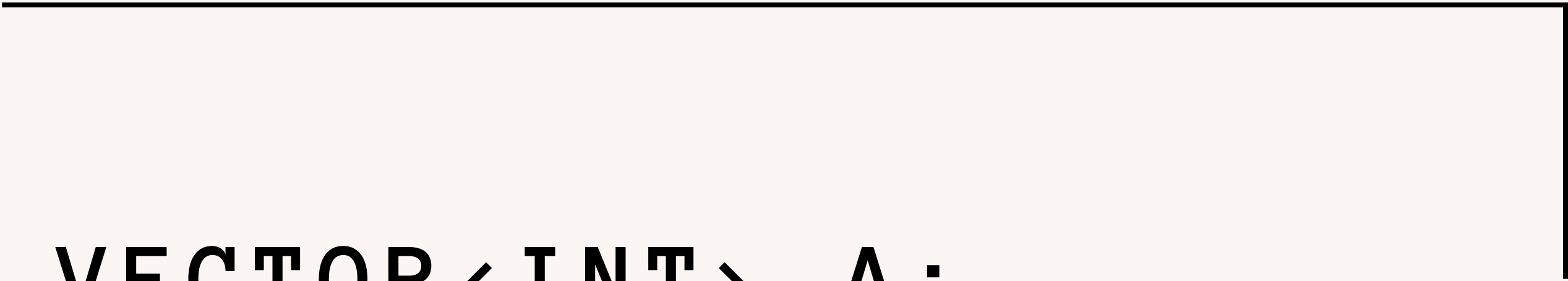
$O (N \ LOG \ N)$



```
VECTOR<INT> A;
```

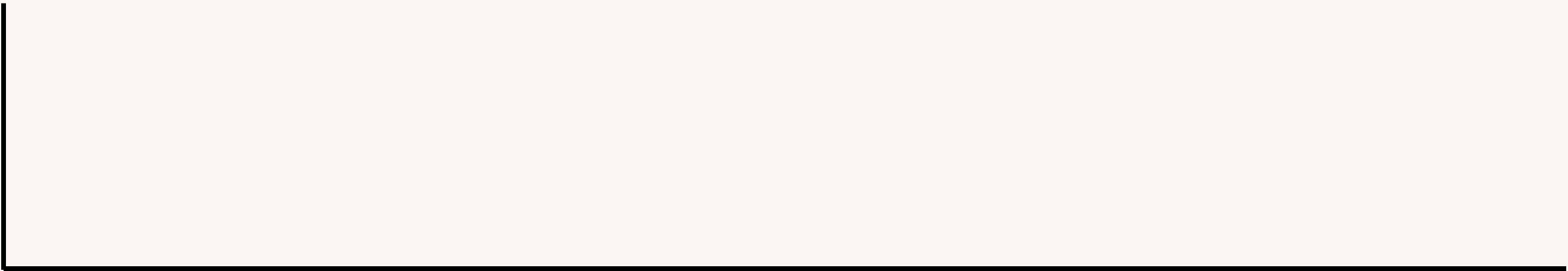
```
INT A[10];
```

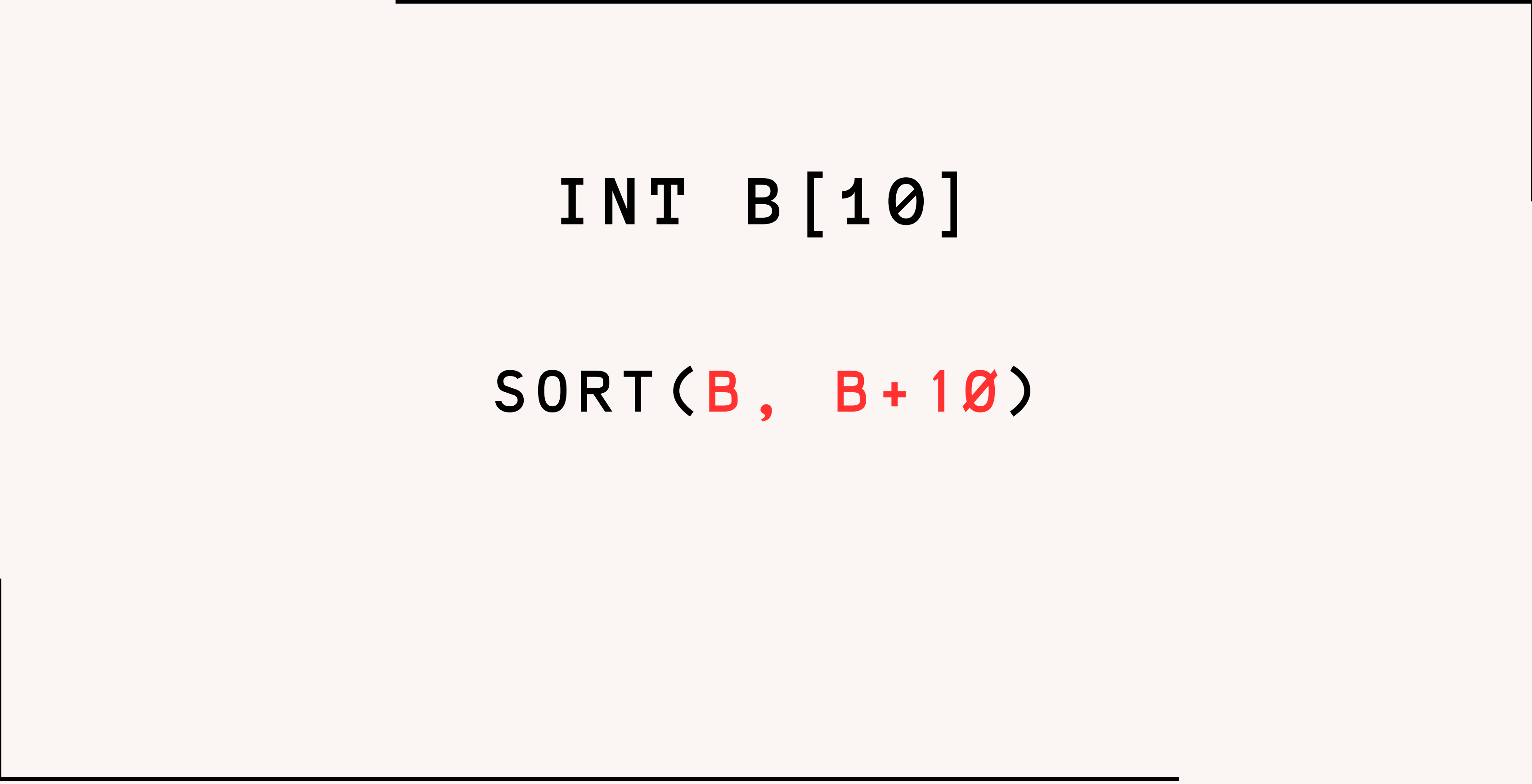




```
VECTOR<INT> A;
```

```
SORT(A.BEGIN(), A.END())
```





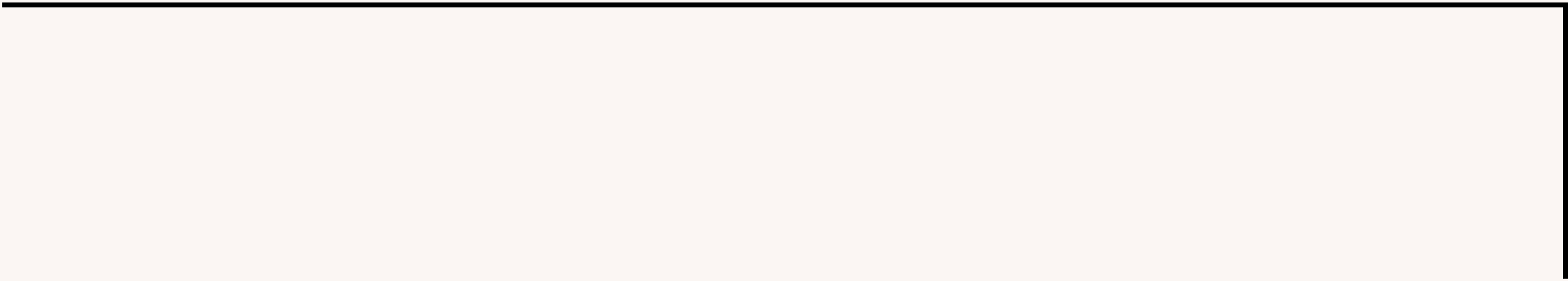
```
INT B[10]
```

```
SORT(B, B + 10)
```

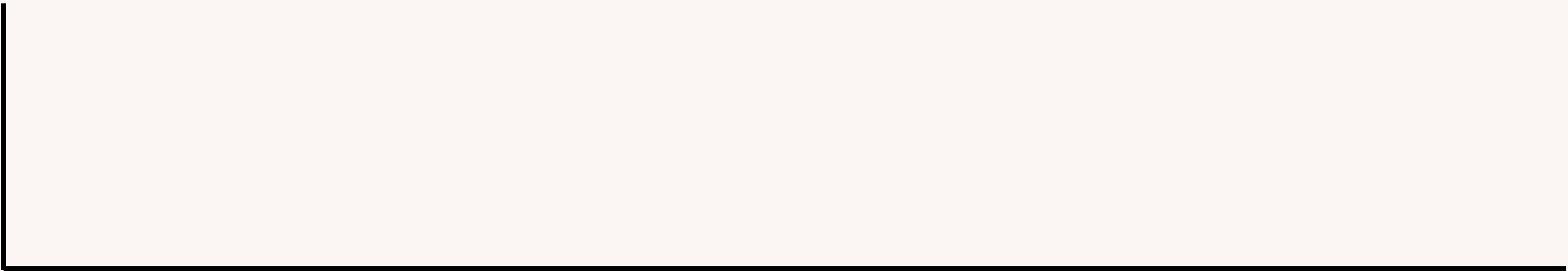
ORDENAÇÃO DO MAIOR PARA O MENOR

```
SORT(A.BEGIN(), A.END(), GREATER<INT>())
```

```
signed main(){
    winton;
    int v;
    cin >> v;
    vector<int> a(3);
    for (auto &i : a) cin >> i;
    sort(all(a));
    int i = 0;
    int ans = 0;
    while(v >= a[i] && i < 3){
        ans++;
        v -= a[i];
        i++;
    }
    cout << ans << endl;
}
```



C . FIGURINHAS DA COPA



Em ano de Copa do Mundo de Futebol, o álbum de figurinhas oficial é sempre um grande sucesso entre crianças e também entre adultos. Para quem não conhece, o álbum contém espaços numerados de 1 a N para colar as figurinhas; cada figurinha, também numerada de 1 a N , é uma pequena foto de um jogador de uma das seleções que jogará a Copa do Mundo. O objetivo é colar todas as figurinhas nos respectivos espaços no álbum, de modo a completar o álbum (ou seja, não deixar nenhum espaço sem a correspondente figurinha).

Algumas figurinhas são *carimbadas* (efetivamente têm um carimbo impresso sobre a fotografia do jogador) e são mais raras, mais difíceis de conseguir.

As figurinhas são vendidas em envelopes fechados, de forma que o comprador não sabe quais figurinhas está comprando, e pode ocorrer de comprar uma figurinha que ele já tenha colado no álbum. Para ajudar os usuários, a empresa responsável pela venda do álbum e das figurinhas quer criar um aplicativo que permita gerenciar facilmente as *figurinhas* que faltam para completar o álbum.

Dados o número total de espaços e figurinhas do álbum (N), a lista das figurinhas carimbadas e uma lista das figurinhas já compradas (que pode conter figurinhas repetidas), sua tarefa é determinar quantas *figurinhas carimbadas* faltam para *completar* o álbum.

Input

A primeira linha contém três números inteiros N , C e M indicando respectivamente o número de figurinhas (e espaços) do álbum, o número de figurinhas carimbadas do álbum e o número de figurinhas já compradas. A segunda linha contém C números inteiros distintos X_i indicando as figurinhas carimbadas do álbum. A terceira linha contém M números inteiros Y_i indicando as figurinhas já compradas.

- $1 \leq N \leq 100$
- $1 \leq C \leq N / 2$
- $1 \leq M \leq 300$
- $1 \leq X_i, Y_i \leq N$

Output

Seu programa deve produzir um inteiro representando o número de figurinhas carimbadas que falta para completar o álbum.

input

10 2 5
4 7
7 1 2 8 3

output

1

input

10 2 6
4 7
7 1 8 4 9 3

output

0

4

Time: 15 ms, memory: 20 KB

Verdict: OK

Input

1 1 4
1
1 1 1 1

Participant's output

0

Jury's answer

0

Checker comment

ok answer is '0'

**VER QUANTOS NÚMEROS ESTÃO
PRESENTES EM AMBAS AS LISTAS**

input

10 2 5

4 7

7 1 2 8 3

output

1

	1	2	3	4	5	6	7	8
CAR =	0	0	0	1	0	0	1	0
TOT =	1	1	1	0	0	0	1	1

RESPOSTA = CARIMBADAS - IGUAIS

```
int car[MAX], tot[MAX];

signed main(){
    winton;
    int n, m, c;
    cin >> n >> c >> m;
    for (int i = 0; i < c; i++){
        int a;
        cin >> a;
        car[a] = 1;
    }
    for (int i = 0; i < m; i++){
        int a;
        cin >> a;
        tot[a] = 1;
    }
    int ans = 0;
    for (int i = 0; i < MAX; i++){
        if (car[i] && tot[i])ans++;
    }
    cout << c - ans << endl;
}
```

D . GARAMANA

Um anagrama de uma palavra é um rearranjo das letras da palavra. Por exemplo,

1. "rota" é um anagrama de "ator";
2. "amor" é um anagrama de "roma"; e
3. os anagramas de "aab" são "aab", "aba" e "baa".

Um anagrama curinga de uma palavra é um anagrama em que algumas das letras podem ter sido substituídas pelo caractere "*" (asterisco). Por exemplo, três possíveis anagramas curingas de "amor" são "*mor", "a**r" e "r**a".

Dadas duas palavras, escreva um programa para determinar se a segunda palavra é um anagrama curinga da primeira palavra.

Input

A primeira linha da entrada contém P ($1 \leq P \leq 100$), a primeira palavra.

A segunda linha contém A ($1 \leq A \leq 100$), a segunda palavra.

As palavras P e A têm o mesmo comprimento, compostas de letras minúsculas não acentuadas. Além disso, A contém caracteres "*" (asterisco), indicando as letras a serem substituídas para formar um anagrama curinga.

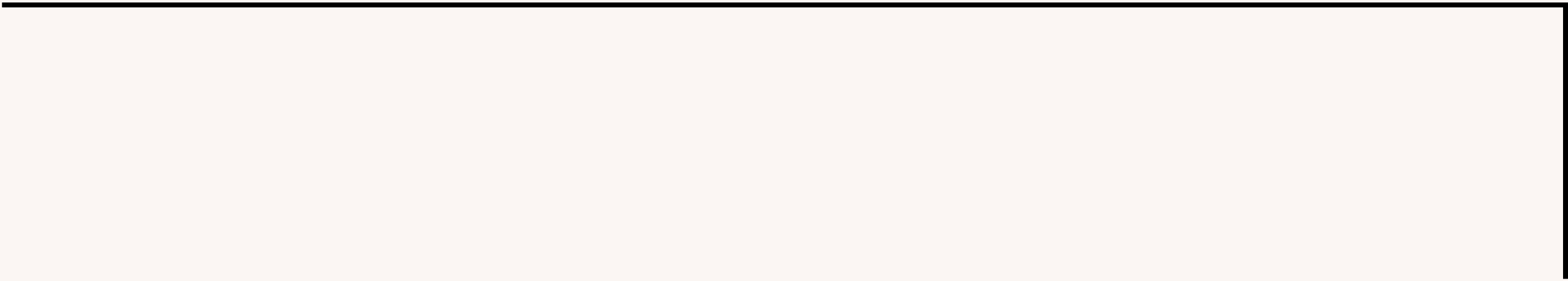
Output

Seu programa deve produzir uma única linha, contendo um único caractere, que deve ser "S" se A é um anagrama curinga de P , ou "N" caso contrário.

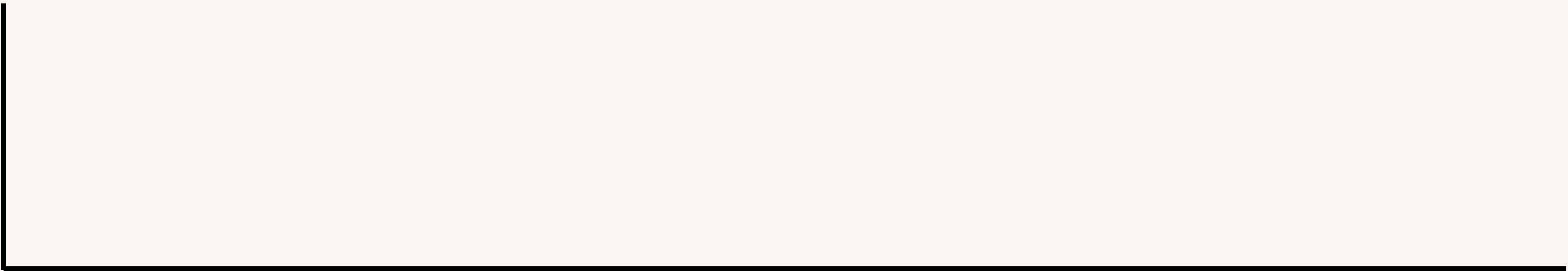
input
olimpiada poliamida
output
S

input
roma ator
output
N

input
microfone *conform*
output
S



**CONTAR A QUANTIDADE DE
CADA LETRA EM CADA PALAVRA**



OLIMPIADA

O : 1

L : 1

I : 2

M : 1

P : 1

A : 2

D : 1

POLIAMIDA

P : 1

O : 1

L : 1

I : 2

A : 2

M : 1

D : 1

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
ROMA	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0
ATOR	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
ROMA	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0
ATOR	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	0	0	0	0

VERIFICAR QUANTAS
LETRAS DIFERENCIAM

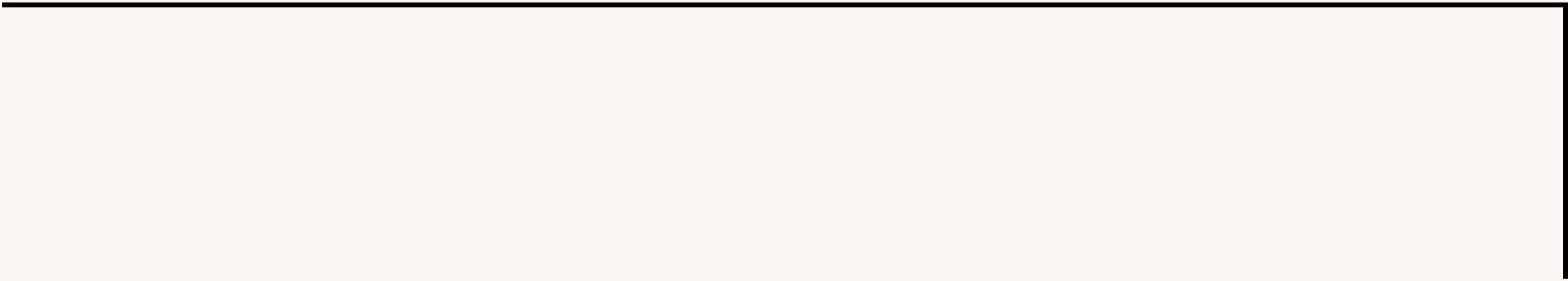


**GUARDAR OS CORINGAS
EM OUTRA VARIÁVEL**

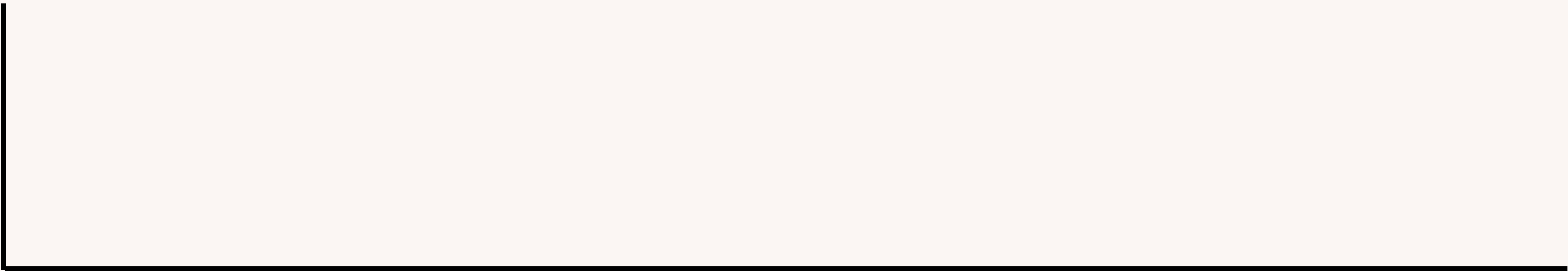
**NUMERO DE DIFERENCAS TEM QUE SER
MENOR QUE DE CORINGAS**



```
signed main(){
    winton;
    string s, t;
    cin >> s >> t;
    vector<int> a1(26);
    vector<int> a2(26);
    int c = 0;
    for(int i = 0; i < s.size(); i++){
        a1[s[i] - 'a']++;
        if (t[i] == '*') c++;
        else a2[t[i] - 'a']++;
    }
    bool ans = true;
    for (int i = 0; i < 26; i++){
        int diff = abs(a1[i] - a2[i]);
        c -= diff;
    }
    if (c != 0) cout << "N" << endl;
    else cout << "S" << endl;
}
```



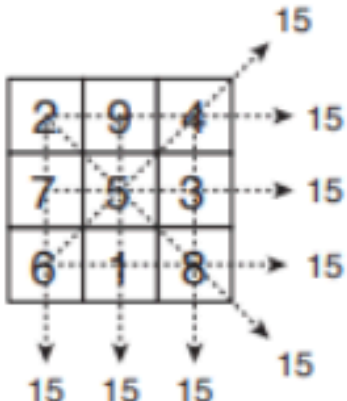
E . QUADRADO MAGICO



Em um Quadrado Mágico, a soma de qualquer coluna, linha ou diagonal tem sempre o mesmo valor, e nenhum número aparece mais do que uma vez.

A dimensão de um quadrado mágico é o número de colunas (ou de linhas, já que o número de colunas é igual ao número de linhas).

2	9	4
7	5	3
6	1	8



The diagram shows a 3x3 magic square with the same numbers as the table above. Arrows point from each row, column, and the two main diagonals to the number 15, indicating that all these sums are equal to 15.

Rita encontrou um caderno antigo de sua avó, repleto de quadrados mágicos de todas as dimensões. Infelizmente alguns dos números estão ilegíveis. Você pode ajudá-la?

Dado um quadrado mágico com exatamente um número ilegível, determine o valor e a posição desse número.

Input

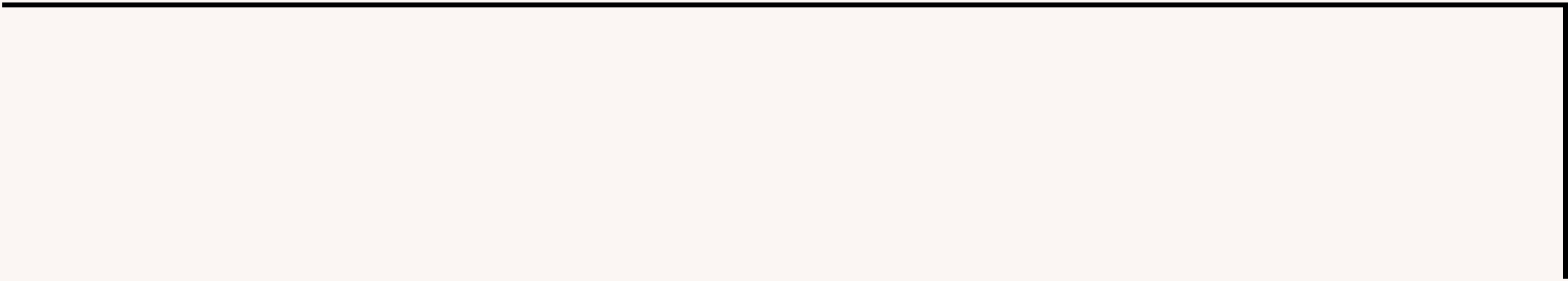
A primeira linha da entrada contém um número inteiro N ($3 \leq N \leq 10$), a dimensão do quadrado mágico. Cada uma das N linhas seguintes contém N inteiros X_i ($0 \leq X_i \leq 100$), para $1 \leq i \leq N$. Exatamente um dos números do quadrado da entrada é igual a zero, indicando o número ilegível.

Output

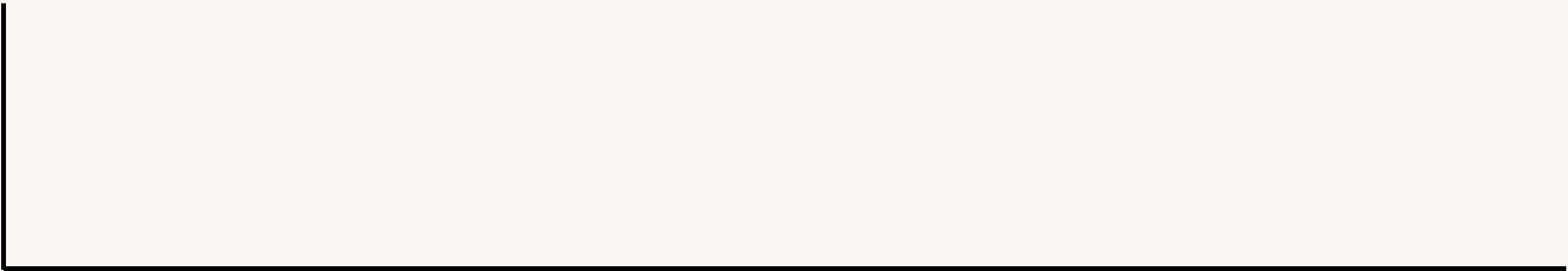
Seu programa deve produzir três linhas, cada uma contendo um único número inteiro. A primeira linha deve conter o valor do número ilegível. A segunda linha deve conter a linha do número ilegível no quadrado (as linhas do quadrado variam de 1 a N). A terceira linha deve conter a coluna do número ilegível no quadrado (as colunas do quadrado variam de 1 a N).

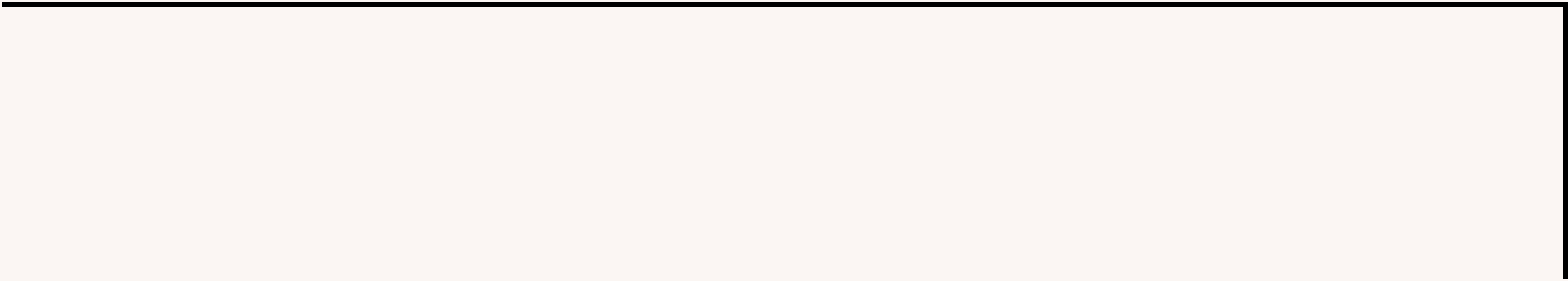
input
3
6 1 8
7 5 3
2 0 4
output
9
3
2

input
4
16 3 2 13
9 6 0 12
5 10 11 8
4 15 14 1
output
7
2
3

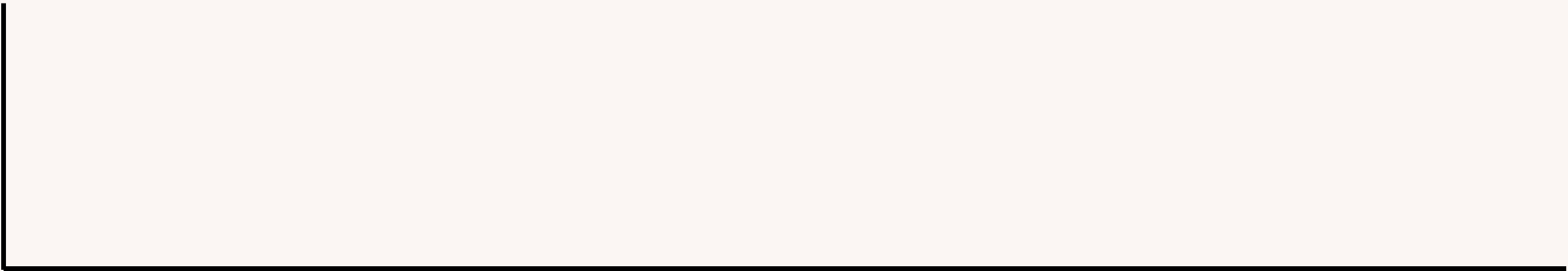


**QUANDO ACHAR O ZERO
GUARDAR SUA POSIÇÃO**





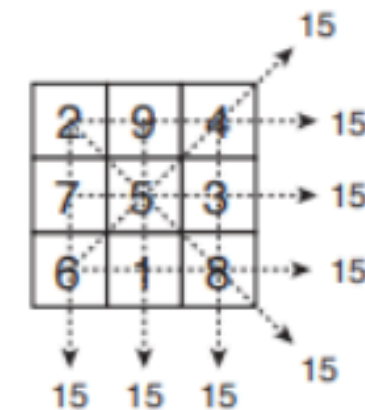
MAS E PARA ACHAR O VALOR?



Em um Quadrado Mágico, a soma de qualquer coluna, linha ou diagonal tem sempre o mesmo valor, e nenhum número aparece mais do que uma vez.

A dimensão de um quadrado mágico é o número de colunas (ou de linhas, já que o número de colunas é igual ao número de linhas).

2	9	4
7	5	3
6	1	8



The diagram shows a 3x3 magic square with the same numbers as the table above. Dashed lines and arrows indicate the sums of rows, columns, and diagonals, all of which equal 15. The first row (2, 9, 4) has a right-pointing arrow labeled 15. The second row (7, 5, 3) has a right-pointing arrow labeled 15. The third row (6, 1, 8) has a right-pointing arrow labeled 15. The first column (2, 7, 6) has a downward arrow labeled 15. The second column (9, 5, 1) has a downward arrow labeled 15. The third column (4, 3, 8) has a downward arrow labeled 15. The main diagonal (2, 5, 8) has a downward-right arrow labeled 15. The anti-diagonal (4, 5, 6) has an upward-right arrow labeled 15.

Rita encontrou um caderno antigo de sua avó, repleto de quadrados mágicos de todas as dimensões. Infelizmente alguns dos números estão ilegíveis. Você pode ajudá-la?

Dado um quadrado mágico com exatamente um número ilegível, determine o valor e a posição desse número.

Input

A primeira linha da entrada contém um número inteiro N ($3 \leq N \leq 10$), a dimensão do quadrado mágico. Cada uma das N linhas seguintes contém N inteiros X_i ($0 \leq X_i \leq 100$), para $1 \leq i \leq N$. Exatamente um dos números do quadrado da entrada é igual a zero, indicando o número ilegível.

Output

Seu programa deve produzir três linhas, cada uma contendo um único número inteiro. A primeira linha deve conter o valor do número ilegível. A segunda linha deve conter a linha do número ilegível no quadrado (as linhas do quadrado variam de 1 a N). A terceira linha deve conter a coluna do número ilegível no quadrado (as colunas do quadrado variam de 1 a N).

É GARANTIDO QUE OS
NÚMEROS VÃO DE 1 ATÉ N^2

MISSING NUMBER

1 2 3 4 5 SOMA = 15

1 2 0 4 5 SOMA = 12

RESPOSTA = 3

```
signed main(){
    winton;
    int n;
    cin >> n;
    int x,y,ans = 0;
    int tot = 0;
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= n; j++){
            int a;
            cin >> a;
            ans += a;
            if (!a){
                x=i;
                y=j;
            }
        }
    }
    for (int i = 1; i<= n*n; i++){
        tot += i;
    }
    cout << tot - ans << endl << x << endl << y << endl;
}
```

F . CHUVA

Está chovendo tanto na Obilândia que começaram a aparecer goteiras dentro da casa do prefeito. Uma dessas goteiras está fazendo escorrer água verticalmente, a partir de um ponto no teto, numa parede onde há várias prateleiras horizontais. Quando a água bate em uma prateleira, ela começa a escorrer horizontalmente para os dois lados, direita e esquerda, até as extremidades da prateleira, quando volta a escorrer verticalmente.

Vamos representar a parede por uma matriz de N linhas e M colunas de caracteres, como mostrado abaixo. As prateleiras serão representadas por "#" e a parede por ".". Só existem prateleiras nas linhas pares e elas nunca encostam na borda da parede. Há apenas um ponto de vazamento representado pelo caractere "o" na primeira linha.

Para deixar mais rigorosa a forma como a água vai escorrer, seja $c(i,j)$ o caractere na linha i coluna j . Se $c(i,j) = "."$, então ele deve virar "o" sempre que:

1. $c(i-1,j) = "o"$; ou
2. $c(i,j-1) = "o"$ e $c(i+1,j-1) = "#"$; ou
3. $c(i,j+1) = "o"$ e $c(i+1,j+1) = "#"$.

Neste problema, dada a matriz representando a parede no início do vazamento, seu programa deve imprimir na saída uma matriz representando a parede usando o caractere "o" exatamente nas posições que serão molhadas pelo vazamento, como ilustrado acima.

Input

A primeira linha do input contém um N ($3 \leq N \leq 500$) e M ($3 \leq M \leq 500$), respectivamente o número de linhas e colunas da matriz. Há exatamente um caractere "o" na primeira linha. As linhas ímpares, a primeira coluna e a última coluna não possuem o caractere "#".

As N linhas seguintes da entrada contêm, cada uma, uma sequência de M caracteres entre três possíveis: ".", "#" ou "o".

Output

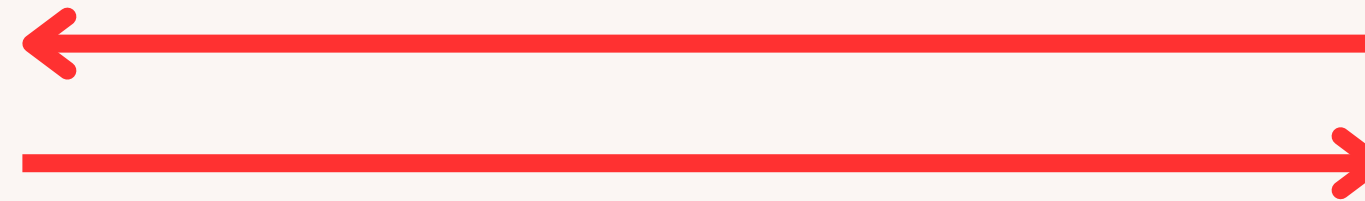
Seu programa deve imprimir N linhas, cada uma contendo uma sequência de M caracteres, representando a matriz da entrada usando o caractere "o" exatamente nas posições que serão molhadas pelo vazamento.

. 0
. ### . . . ##### . # .
.
. #####
.
. # . ##### ## .
.
. #####
.



. 000000 . .
. ### . . 0#####0# .
. 000000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
0#0##### . 0 . 0##0
0 . 0 . 00000000 . . 0
0 . 0 . 0#####00 . . 0
0 . 0 . 0 00 . . 0

**BASTA SIMULAR
LINHA POR LINHA**



. 000000 . .
 . ### . . . ##### . # .

 . . ### ## ## ##

 . # . ## ## ## ## .

 ## ## ##

. 000000 . .
. ### . . 0####0# .
.
. . #####
.
. # . #### . . . ## .
.
. #####
.

. 000000 . .
. ### . . 0#####0# .
. 00000000 . . 0 . .
. . #####
.
. # . ##### . . . ## .
.
. #####
.

. 000000 . .
. ### . . 0####0# .
. 00000000 . . 0 . .
. 0#####0 . . 0 . .

.
. # . ##### . . . ## .
.
. #####
.

. 000000 . .
. ### . . 0####0# .
. 00000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
. # . ##### . . ## .
.
. #####
.

. 000000 . .
. ### . . 0####0# .
. 00000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
0#0#### . 0 . 0##0

.
. #####
.

. 000000 . .
. ### . . 0#####0# .
. 000000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
0#0##### . 0 . 0##0
0 . 0 . 00000000 . . 0
.
.
.

. 000000 . .
. ### . . 0####0# .
. 00000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
0#0#### . 0 . 0##0
0 . 0 . 00000000 . . 0
0 . 0 . 0####00 . . 0
.
.

. 000000 . .
. ### . . 0####0# .
. 00000000 . . 0 . .
. 0#####0 . . 0 . .
000 0 . 0000
0#0#### . 0 . 0##0
0 . 0 . 00000000 . . 0
0 . 0 . 0####00 . . 0
0 . 0 . 0 . . . 00 . . 0

```

char grid[MAX][MAX];

signed main(){
    winton;
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++){
            cin >> grid[i][j];
        }
    }
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++){
            if (grid[i][j] == '#') continue;
            if (grid[i][j] == '.'){
                if (i > 0 && grid[i-1][j] == 'o') grid[i][j] = 'o';
            }
            if (grid[i][j] == 'o'){
                if (i < n+1 && grid[i+1][j] == '#'){
                    if (grid[i][j+1] != '#' && j < m-1) grid[i][j+1] = 'o';
                    if (grid[i][j-1] != '#' && j > 0) grid[i][j-1] = 'o';
                }
            }
        }
    }
    for (int j = m-1; j >= 0; j--){
        if (grid[i][j] == '#') continue;
        if (grid[i][j] == '.'){
            if (i > 0 && grid[i-1][j] == 'o') grid[i][j] = 'o';
        }
        if (grid[i][j] == 'o'){
            if (i >= 0 && grid[i+1][j] == '#'){
                if (grid[i][j+1] != '#' && j < m-1) grid[i][j+1] = 'o';
                if (grid[i][j-1] != '#' && j > 0) grid[i][j-1] = 'o';
            }
        }
    }
}
for (int i = 0; i < n; i++){
    for (int j = 0; j < m; j++){
        cout << grid[i][j];
    }
    cout << endl;
}
}

```

BUSCA BINÁRIA

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10

**QUEREMOS BUSCAR A POSIÇÃO
DO VALOR $x = 7$**

X = 7

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10



L



MID



R

MID > X?

X = 7

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10

L **MID** **R**

MID > X ?



X = 7

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10



L



MID



R

MID > X?

X = 7

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10



L



MID



R

MID > X ?



X = 7

0	1	2	3	4	5	6	7	8	9
-5	-2	0	1	2	4	5	6	7	10

↑ ↑ ↑
L M I D R

M I D == X

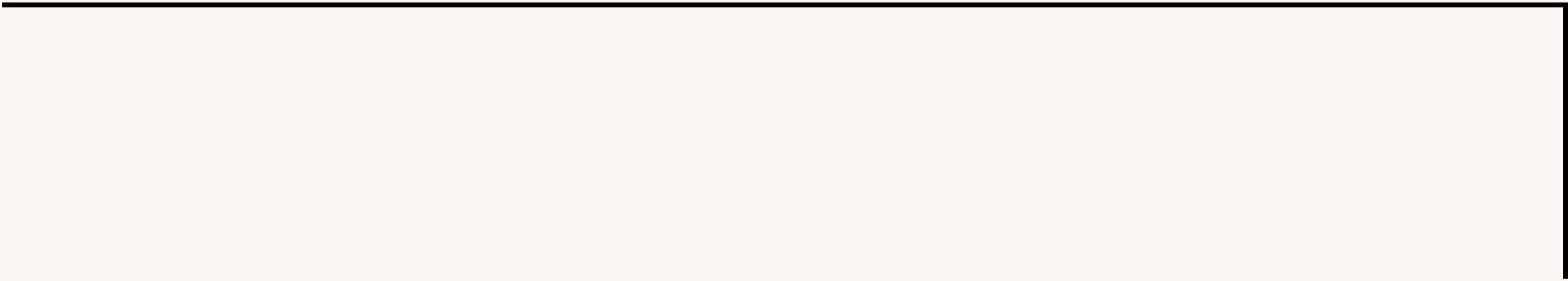
```
int binarySearch(int arr[], int l, int r, int x)
{
    while (l <= r) {
        int mid = (l + r)/2;

        // checar se chegamos na resposta
        if (arr[mid] == x)
            return mid;

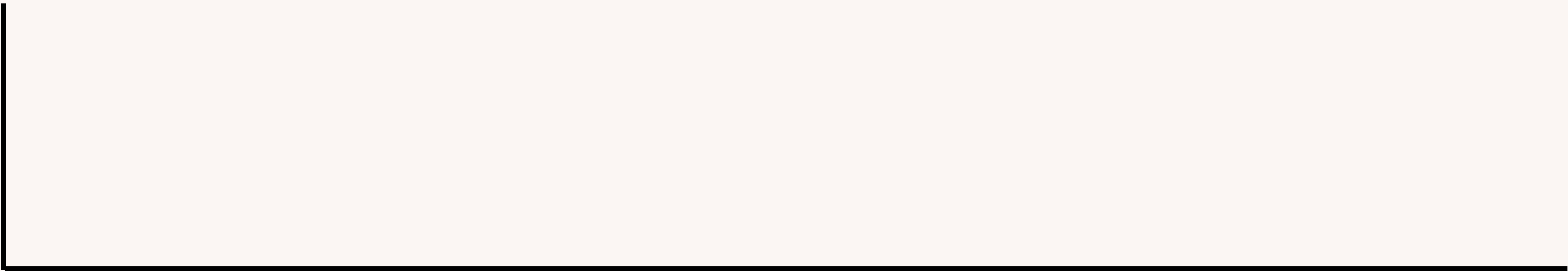
        // se x for maior, ignorar a parte da esquerda
        if (arr[mid] < x)
            l = mid + 1;

        // se o x for menor, ignorar a parte da direita
        else
            r = mid - 1;
    }

    // Se chegarmos aqui, o elemento n estava presente
    return -1;
}
```



BUSCA BINÁRIA NA RESPOSTA



Factory Machines

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s Memory limit: 512 MB

A factory has n machines which can be used to make products. Your goal is to make a total of t products.

For each machine, you know the number of seconds it needs to make a single product. The machines can work simultaneously, and you can freely decide their schedule.

What is the shortest time needed to make t products?

Input

The first input line has two integers n and t : the number of machines and products.

The next line has n integers $k_1, k_2, \dots, k_n$: the time needed to make a product using each machine.

Output

Print one integer: the minimum time needed to make t products.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq t \leq 10^9$
- $1 \leq k_i \leq 10^9$

Example

Input:

3 7
3 2 5

Output:

8

Explanation: Machine 1 makes two products, machine 2 makes four products and machine 3 makes one product.

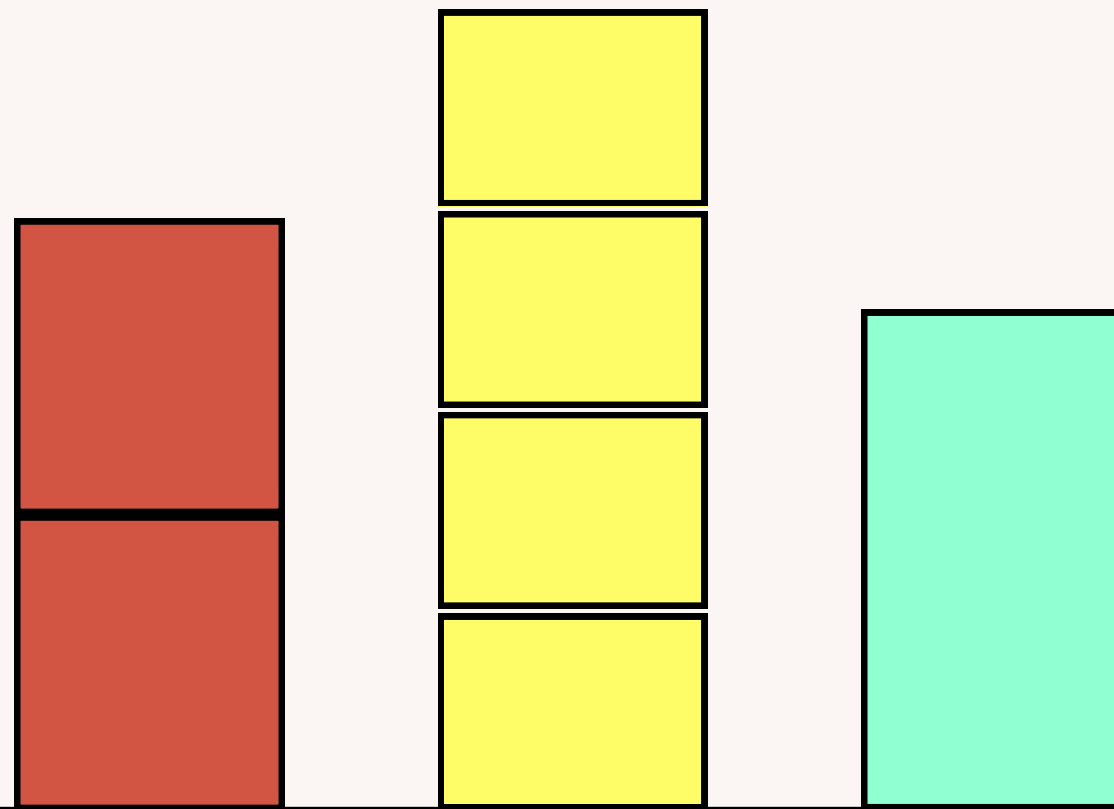
TEMPO

**8
7
6
5
4
3
2
1**

M 1

M 2

M 3



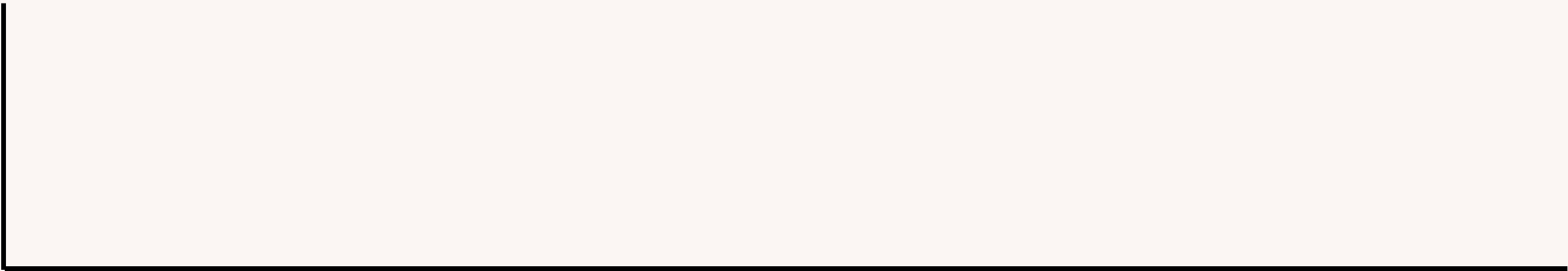
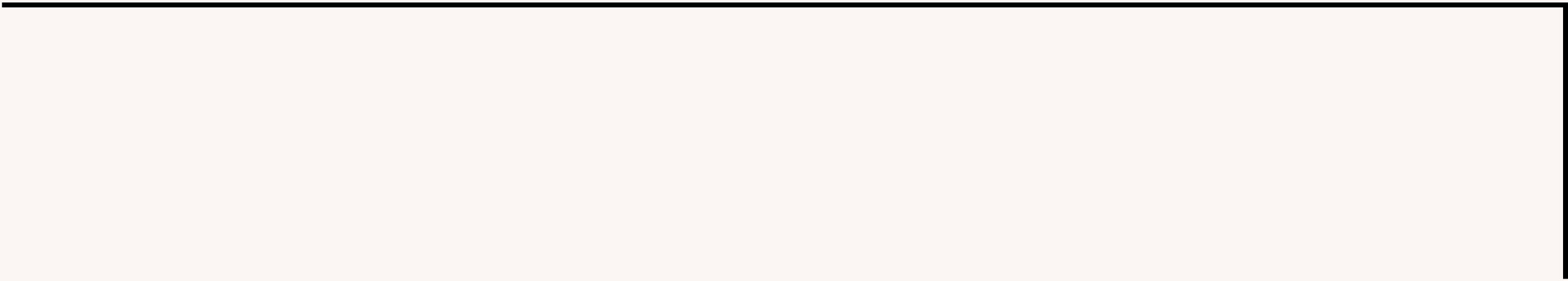
**CONTAR A QUANTIDADE DE
PRODUTOS QUE AS MAQUINAS
CONSEGUEM FAZER EM X MINUTOS**

$$S E X = 15$$

$T1 = 3$	$M1 = 15 / 3 = 5$	} SUM
$T2 = 2$	$M2 = 15 / 2 = 7$	
$T3 = 5$	$M3 = 15 / 5 = 3$	

EM 15 MINUTOS, JUNTAS
FAZEM 15 PRODUTOS

COMO ACHAR O X?



BUSCA BINÁRIA

```
signed main(){
    winton;
    int t, n, mini = INT_MAX;
    cin >> n >> t;
    vector<int> tempos(n);

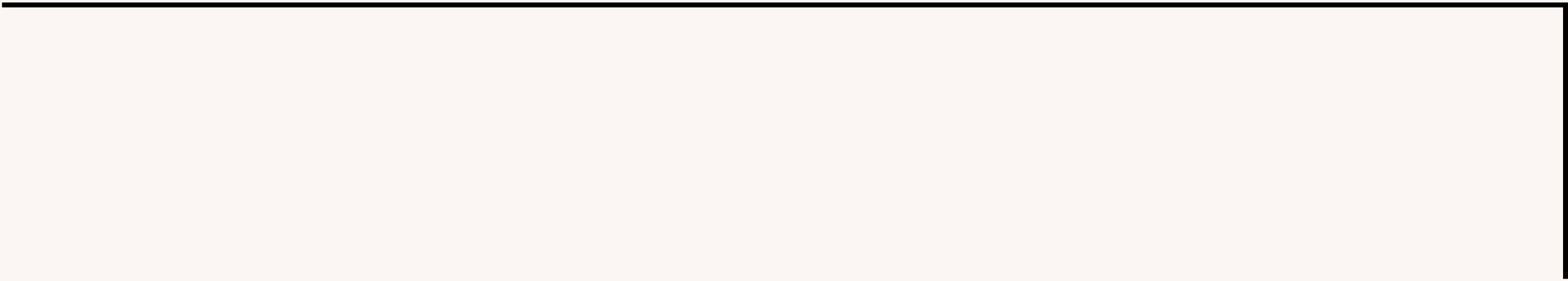
    for (int i = 0; i < n; i++){
        cin >> tempos[i];
        mini = min(mini, tempos[i]);
    }

    int l = 0;
    int r = t*mini;
    int ans = 0;

    while(l<=r){
        int mid = l + (r-l)/2;
        int sum = 0;
        for (int i = 0; i < n; i++){
            sum+=(mid/tempos[i]);
            if (sum >= t) break;
        }

        if (sum>=t){
            ans = mid;
            r = mid-1;
        }
        else {
            l = mid+1;
        }
    }

    cout << ans << endl;
}
```



CSES.FI/PROBLEMSET/MODEL/1620/

